



Nombre: Luis Alberto Vargas González

Fecha: 30/06/2025

Actividad y unidad: U6 A1: Implementar algoritmo de backtracking

Clase: Diseño y Análisis de Algoritmos.

Maestría en Ciberseguridad.

Introducción

La Resolución de sudokus es una actividad que desde muy pequeños se nos enseña a realizar para adquirir agilidad mental pero sobre todo lógica-matemática, y así como hay personas muy habilidosas para poder resolver sudokus tanto simples como complejos en cuestiones de unos pocos minutos o segundos, también existen personas que se les dificulta de una manera muy notoria la resolución de estos ejercicios mentales.

Tomando en cuenta los pasos para resolver tradicionalmente un sudoku, es necesario recalcar que usamos de manera analítica la idea central del backtracking para poder resolver este tipo de ejercicios, aunque muchas veces ni cuenta nos damos de ello, puesto que está en nuestra naturaleza humana el retroceder los pasos que sean necesarios para poder llegar a la meta final de este tipo de ejercicios, que de manera similar ocurre en la manera en que resolvemos las torres de Hanói analíticamente claro está. Por lo que una técnica tan simple pero tan poderosa como esta nos permite llegar al resultado ideal de nuestro objetivo.

Código Fuente.

```
# Función para imprimir el tablero de Sudoku
def imprimir_sudoku(tablero):
    for fila in range(9):
        if fila % 3 == 0 and fila != 0:
            print("-" * 21)
        for col in range(9):
            if col % 3 == 0 and col != 0:
                print("|", end=" ")
            print(tablero[fila][col] if
tablero[fila][col] != 0 else ".", end=" ")
            print()

# Verificar si un número puede colocarse en una
posición
def es_valido(tablero, fila, col, num):
    for i in range(9):
        if tablero[fila][i] == num or tablero[i][col]
== num:
        return False
    start_fila = 3 * (fila // 3)
    start_col = 3 * (col // 3)
    for i in range(start_fila, start_fila + 3):
        for j in range(start_col, start_col + 3):
            if tablero[i][j] == num:
                return False
    return True
```

```

# Algoritmo de backtracking para resolver el Sudoku
def resolver_sudoku(tablero):
    for fila in range(9):
        for col in range(9):
            if tablero[fila][col] == 0:
                for num in range(1, 10):
                    if es_valido(tablero, fila, col,
num):
                        tablero[fila][col] = num
                        if resolver_sudoku(tablero):
                            return True
                        tablero[fila][col] = 0
                return False
    return True

# Sudoku 1 (Imagen 1)
sudoku_1 = [
    [5, 0, 0, 9, 1, 3, 7, 2, 0],
    [3, 0, 0, 0, 8, 0, 5, 0, 9],
    [0, 9, 0, 2, 0, 5, 0, 8, 0],
    [6, 8, 0, 4, 7, 0, 2, 3, 0],
    [0, 0, 9, 5, 0, 0, 4, 6, 0],
    [7, 0, 4, 0, 0, 0, 0, 0, 5],
    [0, 2, 0, 0, 0, 0, 0, 0, 0],
    [4, 0, 0, 8, 9, 1, 6, 0, 0],
    [8, 5, 0, 7, 2, 0, 0, 0, 3]
]

# Sudoku 2 (Imagen 2 corregido)
sudoku_2 = [
    [6, 9, 0, 0, 0, 0, 7, 0, 0],

```

```

    [0, 0, 0, 0, 9, 6, 0, 0, 0],
    [0, 8, 0, 7, 5, 3, 0, 9, 0],
    [0, 2, 0, 3, 7, 4, 5, 6, 1],
    [3, 6, 0, 0, 0, 5, 0, 2, 0],
    [0, 0, 0, 9, 6, 0, 3, 7, 8],
    [0, 0, 6, 0, 3, 1, 0, 8, 4],
    [0, 4, 5, 8, 0, 7, 6, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 5, 7]
]

# Resolver y mostrar Sudoku 1
print("Sudoku 1 - Original:")
imprimir_sudoku(sudoku_1)
if resolver_sudoku(sudoku_1):
    print("\nSudoku 1 - Resuelto:")
    imprimir_sudoku(sudoku_1)
else:
    print("No se pudo resolver Sudoku 1.")

# Resolver y mostrar Sudoku 2
print("\n" + "="*40 + "\n")
print("Sudoku 2 - Original:")
imprimir_sudoku(sudoku_2)
if resolver_sudoku(sudoku_2):
    print("\nSudoku 2 - Resuelto:")
    imprimir_sudoku(sudoku_2)
else:
    print("No se pudo resolver Sudoku 2.")

```

Explicación breve de código.

Tenemos 3 funciones las cuales nos servirán para poder; imprimir, resolver los sudokus , y verificar de manera recursiva si un numero dentro de cada celda no genera conflictos para resolver el sudoku respectivamente.

Función `imprimir_sudoku(tablero)`

Imprime el tablero de Sudoku de manera formateada. Usa puntos para celdas vacías (valor 0) y separa visualmente los bloques de 3x3 con líneas y barras.

Función `es_valido(tablero, fila, col, num)`

Verifica si un número (num) puede colocarse en la posición (fila, col) del tablero:

- Revisa que el número no esté en la misma fila ni columna.
- Revisa que el número no esté en el bloque 3x3 correspondiente.

Función `resolver_sudoku(tablero)`

Implementa el algoritmo de backtracking para resolver el Sudoku:

- Busca la primera celda vacía (valor 0).
- Prueba números del 1 al 9 en esa celda.
- Si el número es válido, lo coloca y llama recursivamente para resolver el resto.
- Si no puede resolver, retrocede (backtracking) y prueba otro número.
- Si llena todas las celdas correctamente, retorna ``True``.

Tableros de Sudoku

Define dos tableros (``sudoku_1`` y ``sudoku_2``) como listas de listas, donde los ceros representan celdas vacías.

. Ejecución principal

- Imprime el Sudoku original.
- Intenta resolverlo usando ``resolver_sudoku``.
- Si se resuelve, imprime el resultado; si no, muestra un mensaje de error.
- Hace esto para ambos tableros.

Resultados

```
PS C:\Users\luisv\Desktop> python backtracking.py
Sudoku 1 - Original:
5 . . | 9 1 3 | 7 2 .
3 . . | . 8 . | 5 . 9
. 9 . | 2 . 5 | . 8 .
-----
6 8 . | 4 7 . | 2 3 .
. . 9 | 5 . . | 4 6 .
7 . 4 | . . . | . . 5
-----
. 2 . | . . . | . . .
4 . . | 8 9 1 | 6 . .
8 5 . | 7 2 . | . . 3

Sudoku 1 - Resuelto:
5 6 8 | 9 1 3 | 7 2 4
3 4 2 | 6 8 7 | 5 1 9
1 9 7 | 2 4 5 | 3 8 6
-----
6 8 5 | 4 7 9 | 2 3 1
2 1 9 | 5 3 8 | 4 6 7
7 3 4 | 1 6 2 | 8 9 5
-----
9 2 6 | 3 5 4 | 1 7 8
4 7 3 | 8 9 1 | 6 5 2
8 5 1 | 7 2 6 | 9 4 3
```

```
=====
Sudoku 2 - Original:
6 9 . | . . . | 7 . .
. . . | . 9 6 | . . .
. 8 . | 7 5 3 | . 9 .
-----
. 2 . | 3 7 4 | 5 6 1
3 6 . | . . 5 | . 2 .
. . . | 9 6 . | 3 7 8
-----
. . 6 | . 3 1 | . 8 4
. 4 5 | 8 . 7 | 6 . .
. . . | . . . | . 5 7

Sudoku 2 - Resuelto:
6 9 3 | 2 1 8 | 7 4 5
7 5 1 | 4 9 6 | 8 3 2
4 8 2 | 7 5 3 | 1 9 6
-----
8 2 9 | 3 7 4 | 5 6 1
3 6 7 | 1 8 5 | 4 2 9
5 1 4 | 9 6 2 | 3 7 8
-----
2 7 6 | 5 3 1 | 9 8 4
9 4 5 | 8 2 7 | 6 1 3
1 3 8 | 6 4 9 | 2 5 7
PS C:\Users\luisv\Desktop>
```


Conclusión.

Podemos argumentar que se realizó con éxito la actividad, si bien fue un poco desafiante el entender como implementar sobre todo la función de backtracking , ya que nunca había hecho académicamente este tipo de implementaciones antes por lo que me tarde un tiempo considerable para poder llegar a la solución adecuada, pues de hecho las primeras versiones de la función no me resolvían ninguno de los dos sudokus, y después solo podía resolver 1 sola, y ya esta última versión del código es la adecuada.

Podemos verificar que para versiones sencillas de sudokus como las que nos fueron proporcionadas, este algoritmo puede resolver rápidamente las celdas a pesar de tener una notación de $O(1)$, debido a todas las restricciones que debe de cumplir por ello la complejidad no crece demasiado a menos de que hablemos de sudokus con tableros gigantes (100 x 100).